

2 - Variables and Basic Data Structures

May 5, 2026

Biến và các kiểu dữ liệu cơ bản

TS. Lê Thành Trung

May 5, 2026

Contents

1	Biến và phép gán	4
1.1	Khái niệm biến	4
1.2	Phép gán và hàm xuất dữ liệu <code>print</code>	4
1.3	Quy tắc đặt tên biến	5
1.4	Gán nhiều biến	5
1.5	Cập nhật giá trị biến	6
1.6	Kiểu dữ liệu của biến	6
1.7	Hàm nhập dữ liệu <code>input</code> và ép kiểu cơ bản	7
1.8	Lỗi thường gặp	7
1.9	Bài tập	7
2	Các toán tử thường gặp	8
2.1	Toán tử số học	8
2.2	Toán tử so sánh	8
2.3	Toán tử logic	9
2.4	Toán tử gán mở rộng	10
2.5	Toán tử kiểm tra phần tử	10
2.6	Toán tử định danh	11
2.7	Ghi nhớ nhanh	11
2.8	Bài tập	11
3	Kiểu dữ liệu <code>String</code>	11
3.1	Khái niệm	11
3.2	Khởi tạo string	12
3.3	Truy cập và cắt chuỗi	12
3.4	Một số phép toán cơ bản	12
3.5	Hàm/phương thức thường dùng	12
3.6	Định dạng chuỗi (f-string)	13
3.7	Lỗi thường gặp	13
3.8	Bài tập	13
4	Kiểu dữ liệu <code>List</code>	14
4.1	Khái niệm	14
4.2	Khởi tạo list	14
4.3	Truy cập phần tử	14
4.4	Thay đổi phần tử	14

4.5	Phương thức thường dùng	15
4.6	Duyệt List	15
4.7	List comprehension (cơ bản)	15
4.8	Lỗi thường gặp	15
4.9	Bài tập	15
5	Kiểu dữ liệu Tuple	16
5.1	Khái niệm	16
5.2	Khởi tạo tuple	16
5.3	Truy cập phần tử	16
5.4	Các phương thức thường dùng	16
5.5	Lỗi thường gặp	17
5.6	Bài tập	17
6	Kiểu dữ liệu Set	17
6.1	Khái niệm	17
6.2	Khởi tạo set	17
6.3	Tính chất không chứa phần tử trùng lặp	17
6.4	Thêm và xóa phần tử	18
6.5	Phép toán tập hợp	18
6.6	Lỗi thường gặp	18
6.7	Bài tập	18
7	Kiểu dữ liệu Dictionary	19
7.1	1. Khái niệm	19
7.2	2. Khởi tạo dictionary	19
7.3	3. Truy cập giá trị	19
7.4	4. Thêm / sửa / xóa phần tử	19
7.5	5. Duyệt dictionary	19
7.6	6. Một số phương thức thường dùng	19
7.7	7. Lỗi thường gặp	19
7.8	8. Bài tập nhanh	20

1 Biến và phép gán

1.1 Khái niệm biến

- Biến là tên đại diện cho một giá trị trong bộ nhớ.
- Trong Python, biến được tạo ra ngay khi bạn gán giá trị lần đầu.
- Python là ngôn ngữ dynamic typing: kiểu dữ liệu được quyết định khi chạy chương trình.

1.2 Phép gán và hàm xuất dữ liệu print

- Dấu = trong Python là phép gán (assignment), không phải phép so sánh.
- Phép so sánh bằng là ==.

```
[1]: x = 10
      name = "An"
      PI = 3.14

      print(x)
      print(name)
      print(PI)
```

```
10
An
3.14
```

```
[2]: print('x = ', x)
      print('name = ', name)
      print('PI = ', PI)
```

```
x = 10
name = An
PI = 3.14
```

```
[3]: print(f'x = {x}')
      print(f'name = {name}')
      print(f'PI = {PI}')
```

```
x = 10
name = An
PI = 3.14
```

```
[4]: x = 3.14159265

      # Làm tròn bằng f-string: {:.nf} (n chữ số thập phân)
      print(f'x = {x:.2f}')      # 3.14
      print(f'x = {x:.4f}')      # 3.1416

      # Định dạng khoa học
      print(f'x = {x:.2e}')      # 3.14e+00
```

```
# Không có chữ số thập phân
print(f'x = {x:.0f}')    # 3
```

```
x = 3.14
x = 3.1416
x = 3.14e+00
x = 3
```

[]:

1.3 Quy tắc đặt tên biến

- Chỉ gồm chữ cái, chữ số và dấu gạch dưới _.
- Không bắt đầu bằng chữ số.
- Phân biệt chữ hoa/chữ thường (age khác Age).
- Không dùng từ khóa của Python (for, if, class, ...).
- Nên đặt tên có ý nghĩa: student_name, total_score.

```
[5]: ho_va_ten = "Lê Thành Trung"
      print(ho_va_ten)

      ho_va_ten_sv1 = "Nguyễn Văn A"
      print(ho_va_ten_sv1)

      age_sv1 = 20
      age_SV1 = 30
      print(age_sv1)
      print(age_SV1)

      ho_va_ten_sv2 = "Trần Thị B"
      print(ho_va_ten_sv2)
```

```
Lê Thành Trung
Nguyễn Văn A
20
30
Trần Thị B
```

1.4 Gán nhiều biến

```
[6]: a = b = 0
      x, y, z = 1, 2, 3
      print(a, b)
      print(x, y, z)
```

```
0 0
1 2 3
```

1.5 Cập nhật giá trị biến

```
[7]: count = 1
count = count + 1
# hoặc
count += 1

print(count)
```

3

1.6 Kiểu dữ liệu của biến

- Python có nhiều kiểu dữ liệu tích hợp sẵn, chia thành các nhóm chính:
 - **Số:** int, float, complex
 - **Văn bản:** str
 - **Logic:** bool (True hoặc False)
 - **Tập hợp có thứ tự:** list ([1, 2, 3]), tuple ((1, 2, 3))
 - **Tập hợp không thứ tự:** set ({1, 2, 3})
 - **Từ điển (Ánh xạ):** dict ({"key": "value"})
 - **Không có giá trị:** NoneType (None)
- Python là ngôn ngữ **dynamic typing**: kiểu dữ liệu được xác định tự động khi gán giá trị, không cần khai báo trước.

```
[8]: so_nguyen = 10
print(type(so_nguyen))
```

<class 'int'>

```
[9]: so_thuc = 3.14
print(type(so_thuc))
```

<class 'float'>

```
[10]: so_phuc = 2 + 3j
print(type(so_phuc))
```

<class 'complex'>

```
[11]: name = "An"
print(type(name))
```

<class 'str'>

```
[12]: logic = True
print(type(logic))
```

<class 'bool'>

1.7 Hàm nhập dữ liệu input và ép kiểu cơ bản

- Hàm `input()` luôn trả về kiểu `str`, cần ép kiểu nếu muốn tính toán.
- Các hàm ép kiểu thường dùng:
 - `int(x)`: chuyển sang số nguyên
 - `float(x)`: chuyển sang số thực
 - `str(x)`: chuyển sang chuỗi
 - `bool(x)`: chuyển sang giá trị logic

```
[13]: # input() luôn trả về str
tuoi_str = input("Nhập tuổi: ")
print(tuoi_str)
print(type(tuoi_str))
```

32

<class 'str'>

```
[14]: # Ép kiểu sang int để tính toán
tuoi_int = int(tuoi_str)
print(type(tuoi_int))
print(f"Năm sau bạn {tuoi_int + 1} tuổi.")
```

<class 'int'>

Năm sau bạn 33 tuổi.

```
[15]: # Ép kiểu sang float
chieu_cao = float(input("Nhập chiều cao (m): "))
print(f"Chiều cao: {chieu_cao} m, kiểu: {type(chieu_cao)}")
```

Chiều cao: 1.85 m, kiểu: <class 'float'>

1.8 Lỗi thường gặp

- Viết `2name = "Lan"` (sai vì bắt đầu bằng số).
- Nhầm `=` và `==` trong điều kiện.
- Đặt tên biến quá ngắn, khó đọc (`a`, `b`, `c`) cho bài toán lớn.

1.9 Bài tập

1. Tạo 3 biến: tên, tuổi, điểm trung bình.
2. In ra thông tin theo mẫu: Tên: ..., Tuổi: ..., Điểm:
3. Tăng tuổi thêm 1 và in lại kết quả.

```
[16]: ten = "Lê Thành Trung"
tuoi = 20
diem_trung_binh = 8.5
print(f'Tên: {ten}, Tuổi: {tuoi}, Điểm trung bình: {diem_trung_binh}')

tuoi += 1
```

```
print(f'Tên: {ten}, Tuổi: {tuoi}, Điểm trung bình: {diem_trung_binh}')
```

Tên: Lê Thành Trung, Tuổi: 20, Điểm trung bình: 8.5

Tên: Lê Thành Trung, Tuổi: 21, Điểm trung bình: 8.5

2 Các toán tử thường gặp

2.1 Toán tử số học

- Dùng để thực hiện các phép tính cơ bản trên số.
- $x + y$: cộng
- $x - y$: trừ
- $x * y$: nhân
- x / y : chia lấy số thực
- $x // y$: chia lấy phần nguyên
- $x \% y$: chia lấy dư
- $x ** y$: lũy thừa

```
[17]: x = 12
      y = 5
      print(f'x + y = {x + y}')
      print(f'x - y = {x - y}')
      print(f'x * y = {x * y}')
      print(f'x / y = {x / y}')
      print(f'x // y = {x // y}')
      print(f'x % y = {x % y}')
      print(f'x^y = {x ** y}')
```

$x + y = 17$

$x - y = 7$

$x * y = 60$

$x / y = 2.4$

$x // y = 2$

$x \% y = 2$

$x^y = 248832$

2.2 Toán tử so sánh

- Kết quả của các phép so sánh là **True** hoặc **False**.
- $==$: bằng
- $!=$: khác
- $>$: lớn hơn
- $<$: nhỏ hơn

- $\geq$: lớn hơn hoặc bằng
- $\leq$: nhỏ hơn hoặc bằng

```
[18]: x = 3
      y = 3

      print(x == y)

      print(x != y)
```

True
False

- Chú ý: khi so sánh bằng hai số thực thì không nên dùng toán tử `==`.

```
[19]: # Ví dụ sai số số thực: về mặt toán học là bằng nhau, nhưng == có thể cho False
      so_thuc_1 = 0.1 + 0.2
      so_thuc_2 = 0.3
      print(so_thuc_1 == so_thuc_2)

      print(so_thuc_1)
      print(so_thuc_2)
      print(abs(so_thuc_1 - so_thuc_2) < 1e-9)
```

False
0.30000000000000004
0.3
True

```
[20]: # Kết hợp so sánh

      x = 10
      print(5 <= x <= 15) # True
```

True

2.3 Toán tử logic

- Thường dùng trong câu lệnh điều kiện `if`, `while`.
- `and`: đúng khi cả hai vế đều đúng
- `or`: đúng khi ít nhất một vế đúng
- `not`: phủ định giá trị logic

```
[21]: age = 20
      print(18 <= age and age <= 25)
```

True

```
[22]: age = 20
      print(age <= 6 or age >=18)
```

True

```
[23]: age = 20
      print(not (18 <= age and age <= 25))
```

False

2.4 Toán tử gán mở rộng

- Giúp viết ngắn gọn hơn khi cập nhật giá trị biến.
- +=: cộng rồi gán
- -=: trừ rồi gán
- *=: nhân rồi gán
- /=: chia rồi gán
- //=: chia nguyên rồi gán
- %=: chia dư rồi gán
- **=: lũy thừa rồi gán

```
[24]: a = 5
      a += 2 # tương đương a = a + 2
      print(a)
```

7

```
[25]: so_luong_phan_tu = 10
      so_luong_phan_tu = so_luong_phan_tu - 1
      print(so_luong_phan_tu)

      so_luong_phan_tu *= 2
      print(so_luong_phan_tu)
```

9

18

2.5 Toán tử kiểm tra phần tử

- Dùng để kiểm tra một giá trị có thuộc một chuỗi, list, tuple, set hay không.
- in: có tồn tại
- not in: không tồn tại

```
[26]: # Kiểm tra phần tử trong danh sách
      names = ["An", "Binh", "Chi"]
```

```
print("An" in names)
print("Dung" not in names)
```

True

True

2.6 Toán tử định danh

- Dùng để kiểm tra hai biến có đang tham chiếu đến cùng một đối tượng trong bộ nhớ hay không.
- `is`: cùng đối tượng
- `is not`: khác đối tượng

Lưu ý: - `is` khác với `==`. - `==` so sánh giá trị. - `is` so sánh danh tính đối tượng.

```
[27]: a = [1, 2, 3]
      b = [1, 2, 3]
      c = a

      print(a == b)  # True: cùng giá trị
      print(a is b)  # False: khác đối tượng trong bộ nhớ
      print(a is c)  # True: cùng tham chiếu một đối tượng
```

True

False

True

2.7 Ghi nhớ nhanh

- Nhóm số học: tính toán.
- Nhóm so sánh: trả về `True` hoặc `False`.
- Nhóm logic: kết hợp các điều kiện.
- Nhóm gán mở rộng: cập nhật biến ngắn gọn.
- Nhóm kiểm tra phần tử: kiểm tra phần tử có thuộc tập dữ liệu.
- Nhóm định danh: kiểm tra hai biến có cùng đối tượng hay không.

2.8 Bài tập

1. Viết biểu thức kiểm tra một học sinh có tuổi từ 18 đến 22 hay không.
2. Kiểm tra ký tự "a" có xuất hiện trong chuỗi "python basics" hay không.

3 Kiểu dữ liệu String

3.1 Khái niệm

- `String` là chuỗi ký tự, đặt trong dấu nháy đơn `'...'` hoặc nháy kép `"..."`.
- Không thể thay đổi trực tiếp từng ký tự của biến kiểu `String` trong Python.

3.2 Khởi tạo string

```
[28]: s1 = 'Hello'
      s2 = "Python"
      s3 = """Chuỗi
      nhiều dòng"""
      print(s1)
      print(s2)
      print(s3)
```

Hello
Python
Chuỗi
nhiều dòng

3.3 Truy cập và cắt chuỗi

```
[29]: text = "Hello, World!"
      print(text[0])
      print(text[1])
      print(text[-1])
      print(text[-2])
      print(text[0:5])  # Hello
```

H
e
!
d
Hello

3.4 Một số phép toán cơ bản

```
[30]: a = "hello"
      b = "world"
      print(a + " " + b)  # hello world

      print(a*3)
```

hello world
hellohellohello

3.5 Hàm/phương thức thường dùng

```
[31]: our_string = "  hoc Python co ban  "
      print(our_string)
      print("strip:", our_string.strip())
      print("upper:", our_string.upper())
      print("lower:", our_string.lower())
```

```
print("replace:", our_string.replace("co ban", "nang cao"))
print("split:", our_string.split())
```

```
hoc Python co ban
strip: hoc Python co ban
upper:  HOC PYTHON CO BAN
lower:  hoc python co ban
replace: hoc Python nang cao
split: ['hoc', 'Python', 'co', 'ban']
```

3.6 Định dạng chuỗi (f-string)

```
[32]: name = "Minh"
      age = 20
      name_and_age = f"Ten: {name}, Tuổi: {age}"
      print(name_and_age)
```

Ten: Minh, Tuổi: 20

3.7 Lỗi thường gặp

- Truy cập vượt chỉ số (IndexError).
- Quên escape ký tự đặc biệt trong chuỗi.
- Cố gắng sửa trực tiếp ký tự: `s[0] = 'H'` (sai với string).

```
[33]: text = "Hello, World!"
      len_text = len(text)
      print(f'Độ dài của chuỗi "{text}" là: {len_text}')

      print(text[len_text - 1])
```

Độ dài của chuỗi "Hello, World!" là: 13
!

```
[34]: text[0] = 'h' # Lỗi: chuỗi là bất biến (immutable)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[34], line 1
----> 1 text[0] = 'h' # Lỗi: chuỗi là bất biến (immutable)

TypeError: 'str' object does not support item assignment
```

3.8 Bài tập

1. Nhập họ tên, in ra chữ hoa toàn bộ.
2. Đếm số ký tự của chuỗi.
3. Tách họ và tên từ chuỗi họ tên.

```
[ ]: ho_va_ten = "Lê Thành Trung"
len_ho_va_ten = len(ho_va_ten)
print(f'Độ dài của chuỗi "{ho_va_ten}" là: {len_ho_va_ten}')
print("split:", ho_va_ten.split())
```

Độ dài của chuỗi "Lê Thành Trung" là: 14
split: ['Lê', 'Thành', 'Trung']

4 Kiểu dữ liệu List

4.1 Khái niệm

- List là cấu trúc dữ liệu có thứ tự (ordered), thay đổi được (mutable), và cho phép phần tử trùng lặp.
- List có thể chứa nhiều kiểu dữ liệu khác nhau.

4.2 Khởi tạo list

```
[36]: numbers = [1, 2, 3, 4]
info = ["An", 20, 8.5, True]
empty_list = []

print(type(numbers))
print(type(info))
print(type(empty_list))
```

```
<class 'list'>
<class 'list'>
<class 'list'>
```

4.3 Truy cập phần tử

```
[38]: a_list = [10, 20, 30, 40]
print(a_list[0])
print(a_list[-1])
print(a_list[1:3]) # Cắt list từ index 1 đến 2 (không bao gồm index 3)
```

```
10
40
[20, 30]
```

4.4 Thay đổi phần tử

```
[39]: a_list = [1, 2, 3]
a_list[1] = 200
print(a_list)
```

```
[1, 200, 3]
```

4.5 Phương thức thường dùng

```
[51]: a_list = [1, 2, 3]
a_list.append(4)           # thêm cuối
a_list.insert(1, 99)       # chèn tại vị trí
a_list.remove(2)           # xóa theo giá trị
x = a_list.pop()           # lấy và xóa phần tử cuối
a_list.sort()              # sắp xếp tăng dần
a_list.reverse()           # đảo ngược
a_list.extend([5, 6])      # mở rộng list bằng một list khác
print(a_list)
a_list.clear()             # xóa tất cả phần tử
print(a_list)

print(1 in a_list)
```

[99, 3, 1, 5, 6]

[]

False

4.6 Duyệt List

```
[41]: students = ["An", "Binh", "Chi"]
for s in students:
    print(s)
```

An

Binh

Chi

4.7 List comprehension (cơ bản)

```
[42]: squares = [x*x for x in range(1, 6)]
print(type(squares))
print(squares)
```

<class 'list'>

[1, 4, 9, 16, 25]

4.8 Lỗi thường gặp

- Dùng chỉ số vượt phạm vi (`IndexError`).
- Gọi `remove(value)` với giá trị không tồn tại (`ValueError`).
- Nhầm giữa `append()` và `extend()`.

4.9 Bài tập

1. Tạo list 5 số nguyên và tính tổng.
2. Tìm số lớn nhất, nhỏ nhất.

5 Kiểu dữ liệu Tuple

5.1 Khái niệm

- Tuple là tập hợp có thứ tự (ordered), nhưng không thể thay đổi sau khi tạo (immutable).
- Dùng khi dữ liệu cần cố định, tránh bị chỉnh sửa ngoài ý muốn.

5.2 Khởi tạo tuple

```
[48]: tuple_1 = (1, 2, 3)
      tuple_2 = ("An", 20, 8.5)
      single_element_tuple = (5,)      # tuple 1 phần tử cần dấu phẩy
      print(type(tuple_1))
      print(type(tuple_2))
      print(type(single_element_tuple))
```

```
<class 'tuple'>
<class 'tuple'>
<class 'tuple'>
```

5.3 Truy cập phần tử

```
[49]: a_tuple = (10, 20, 30, 40)
      print(a_tuple)
      print(a_tuple[0])
      print(a_tuple[-1])
      print(a_tuple[1:3])
```

```
(10, 20, 30, 40)
10
40
(20, 30)
```

5.4 Các phương thức thường dùng

```
[52]: tuple_a = (1, 2, 3)
      tuple_b = (4, 5)
      print(tuple_a + tuple_b)
      print(tuple_a * 2)
      print(2 in tuple_a)
      print(len(tuple_a))
```

```
(1, 2, 3, 4, 5)
(1, 2, 3, 1, 2, 3)
True
3
```

```
[53]: tuple_t = (1, 2, 2, 3)
      print(tuple_t.count(2))
```



```
print(tuple_t.index(3))
```

```
2
3
```

```
[54]: #Packing và unpacking
point = (10, 20)    # packing
x, y = point        # unpacking
print(x, y)
```

```
10 20
```

5.5 Lỗi thường gặp

- Quên dấu phẩy khi tạo tuple 1 phần tử.
- Cố sửa tuple: `t[0] = 100` (TypeError).

5.6 Bài tập

1. Tạo tuple chứa 5 số.
2. In phần tử đầu, cuối.
3. Đếm số lần xuất hiện của một số bất kỳ.

6 Kiểu dữ liệu Set

6.1 Khái niệm

- Set là tập hợp không có thứ tự (unordered), không chứa phần tử trùng lặp.
- Dùng tốt cho bài toán loại bỏ trùng, kiểm tra membership nhanh.

6.2 Khởi tạo set

```
[55]: a_set = {1, 2, 3}
empty_set = set() # không dùng {} vì {} là dict
print(type(a_set))
print(type(empty_set))
```

```
<class 'set'>
<class 'set'>
```

6.3 Tính chất không chứa phần tử trùng lặp

```
[56]: a_set = {1, 2, 2, 3}
print(a_set)
```

```
{1, 2, 3}
```

6.4 Thêm và xóa phần tử

- Chú ý: set không hỗ trợ truy cập bằng index

```
[62]: a_set = {1, 2, 2, 3}
      print(a_set[0]) # Lỗi: set không hỗ trợ indexing
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[62], line 2
      1 a_set = {1, 2, 2, 3}
----> 2 print(a_set[0]) # Lỗi: set không hỗ trợ indexing

TypeError: 'set' object is not subscriptable
```

```
[66]: a_set = {1, 2, 3}
      a_set.add(4)
      a_set.remove(2) # lỗi nếu không tồn tại
      a_set.discard(10) # không lỗi nếu không tồn tại
      x = a_set.pop() # lấy ngẫu nhiên 1 phần tử (xóa phần tử đó trong set)
      print(a_set)
      print(x)
```

```
{3, 4}
1
```

6.5 Phép toán tập hợp

```
[68]: set_a = {1, 2, 3}
      set_b = {3, 4, 5}
      print(set_a | set_b) # union
      print(set_a & set_b) # intersection
      print(set_a - set_b) # difference
      print(set_a ^ set_b) # symmetric difference
```

```
{1, 2, 3, 4, 5}
{3}
{1, 2}
{1, 2, 4, 5}
```

6.6 Lỗi thường gặp

- Dùng {} để tạo set rỗng.
- Dùng phần tử mutable (ví dụ list) trong set.
- Truy cập phần tử set theo chỉ số (không hỗ trợ).

6.7 Bài tập

1. Nhập danh sách điểm, loại bỏ điểm trùng.

2. Cho 2 tập sinh viên, tìm sinh viên học cả 2 lớp.
3. Tìm sinh viên chỉ thuộc lớp A mà không thuộc lớp B.

7 Kiểu dữ liệu Dictionary

7.1 1. Khái niệm

- Dictionary lưu dữ liệu theo cặp **key: value**.
- Key là duy nhất, immutable (string, number, tuple...).
- Value có thể là bất kỳ kiểu dữ liệu nào.

7.2 2. Khởi tạo dictionary

```
student = {  
    "name": "An",  
    "age": 20,  
    "gpa": 3.5  
}  
empty_dict = {}
```

7.3 3. Truy cập giá trị

```
print(student["name"])  
print(student.get("email"))           # None nếu không có key  
print(student.get("email", "N/A"))
```

7.4 4. Thêm / sửa / xóa phần tử

```
student["major"] = "Computer Science"  # thêm  
student["age"] = 21                    # sửa  
student.pop("gpa")                     # xóa theo key
```

7.5 5. Duyệt dictionary

```
for k in student:  
    print(k, student[k])  
  
for k, v in student.items():  
    print(k, v)
```

7.6 6. Một số phương thức thường dùng

```
print(student.keys())  
print(student.values())  
print(student.items())  
student.update({"city": "HCM"})
```

7.7 7. Lỗi thường gặp

- Truy cập key không tồn tại bằng `dict[key]` gây `KeyError`.

- Dùng kiểu mutable làm key (ví dụ list) gây lỗi.
- Nhầm dictionary với list khi truy cập dữ liệu.

7.8 8. Bài tập nhanh

1. Tạo dictionary thông tin 1 môn học (mã, tên, số tín chỉ).
2. In toàn bộ key và value.
3. Cập nhật số tín chỉ và thêm giảng viên phụ trách.